Software Requirements Specification

for

CyberSentinel

Version 1.0 approved

Prepared by Garima Gayakwad
Krisha Gangaramani
Tanishka Gharwade

Sardar Patel Institute of Technology
Group 3

September 25, 2026

Table of Contents

Table of Contents ii
Revision History ii
1. Introduction 1
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 1
1.5 References 1
2. Overall Description 2
2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Classes and Characteristics 2
2.4 Operating Environment 2
2.5 Design and Implementation Constraints 2
2.6 User Documentation 2
2.7 Assumptions and Dependencies 3
3. External Interface Requirements 3
3.1 User Interfaces 3
3.2 Hardware Interfaces 3
3.3 Software Interfaces 3
3.4 Communications Interfaces 3
4. System Features 4
4.1 System Feature 1 4
4.2 System Feature 2 (and so on) 4
5. Other Nonfunctional Requirements 4
5.1 Performance Requirements 4
5.2 Safety Requirements 5
5.3 Security Requirements 5
5.4 Software Quality Attributes 5
5.5 Business Rules 5
6. Other Requirements 5
Appendix A: Glossary 5
Appendix B: Analysis Models 5
Appendix C: To Be Determined List 6

Revision History

Name	Date	Reason For Changes	Version
Garima Gayakwad, Krisha Gangaramani, Tanishka Gharwade	25/09/2026	Initial Draft	1.0 Draft

1. Introduction

1.1 Purpose

*The purpose of this Software Requirements Specification (SRS) is to meticulously define and document the comprehensive functional and nonfunctional requirements for the **CyberSentinel** project. CyberSentinel is a sophisticated, real-time Machine Learning-Based Network Intrusion Detection and Automated Prevention System (IPS). This document serves as the definitive blueprint of the product's capabilities, detailing the intricate interplay between its local network processing engine and its web-based visualization dashboard. The scope of this specification encompasses the entire end-to-end operational pipeline: beginning with the interception of inbound network traffic on the host machine, progressing through mathematical feature extraction and predictive classification utilizing a pre-trained machine learning model, and culminating in active mitigation strategies executed directly against the host operating system's firewall. Furthermore, this document exhaustively details the communication bridging mechanisms- specifically real-time data streaming and remote cloud integrations- that power the modern command-center dashboard utilized by users to monitor and interpret network threats in real time.*

1.2 Document Conventions

This specification adheres to a set of standardized document conventions to ensure clarity, consistency, and traceability throughout the text.

- ***Formatting: Bold text** is utilized strictly to emphasize critical architectural components, key configuration parameters, and primary user interface elements.*

- ***Requirement Identification:** Every specific, testable functional requirement is prefixed with a unique, hierarchical alphanumeric identifier (e.g., **REQ-F1-01**). This convention guarantees strict traceability from the documented requirement directly back to the observable behavior of the product.*

- ***Priority Ratings:** Requirements are assigned a priority rating of **High**, **Medium**, or **Low**. In the context of CyberSentinel, a **High** priority indicates a core operational requirement without which the intrusion prevention system would fundamentally fail to operate (e.g., traffic monitoring, machine learning inference, and firewall blocking). **Medium** priority indicates crucial supportive infrastructure that enhances the system but does not break core local protection (e.g., cloud logging and live broadcasting). **Low** priority indicates auxiliary user interface features (e.g., the static AI explanation panels).*

- ***Absence of Data:** The explicit placeholder **"TBD" (To Be Determined)** is utilized exclusively in instances where a standard SRS template section demands information that is verifiably absent or entirely unspecified within the current factual implementation of the product.*

1.3 Intended Audience and Reading Suggestions

This comprehensive document is crafted for a diverse audience, each of whom plays a vital role in evaluating, deploying, or utilizing the CyberSentinel platform.

- ***System Evaluators and Stakeholders:*** *It is recommended that stakeholders focus their initial reading on* ***Section 2 (Overall Description)*** *and* ***Section 4 (System Features)****. These sections provide exhaustive, paragraph-level breakdowns of the system's capabilities, explaining how the product autonomously intercepts traffic, identifies threats, and enforces network security without requiring deep technical knowledge of the underlying codebase.*

- ***UI/UX Designers and Product Managers:*** *Personnel responsible for the user experience should prioritize* ***Section 3.1 (User Interfaces)*** *and* ***Section 4.7 (Command-Center Dashboard)****, which meticulously detail the aesthetic constraints, interactive topologies, and visual data rendering capabilities of the web dashboard.*

- ***Security Analysts and Network Administrators:*** *Personnel responsible for operating the system should review* ***Section 5 (Other Nonfunctional Requirements)*** *and* ***Section 2.4 (Operating Environment)*** *to fully comprehend the system's operational boundaries, its reliance on administrative privileges, and the specific safeguard rules implemented to prevent accidental self-blocking of critical network infrastructure.*

- ***Academic Mentors and Reviewers:*** *Reviewers should read the document sequentially, utilizing the detailed paragraphs to validate the structural integrity and factual implementation of the product against its stated architectural goals.*

1.4 Product Scope

CyberSentinel operates as a fully integrated, real-time Intrusion Prevention System (IPS) designed to secure host machines against a wide array of network-based cyber threats. The core objective of the product is to seamlessly transition from passive network monitoring to active, automated defense without requiring human intervention for the initial mitigation step.

The scope of the implemented system begins at the network interface layer, where the processing engine continuously monitors inbound network traffic. Rather than analyzing individual packets in isolation, the system intelligently aggregates this traffic into distinct temporal and volumetric flows, allowing for the calculation of complex statistical features. These features are subsequently evaluated by a predictive machine learning model capable of identifying anomalous patterns characteristic of numerous distinct attack categories.

Crucially, the scope extends beyond mere detection. Upon verifying a high-confidence threat that meets specific thresholds, the system automatically configures the host operating system's firewall, generating rules to instantly sever the attacker's connection. Simultaneously, the system manages a local data stream to push these critical security events directly to a visually rich web dashboard. This dashboard provides security personnel with a comprehensive command center, featuring interactive network topology graphs, real-time protocol distribution charts, and a dedicated Artificial Intelligence analyst panel that translates complex statistical anomalies into clear, actionable, plain-English narratives. The product ultimately empowers administrators to review these automated actions and manually intervene, such as restoring access for false positives, directly from the graphical interface.

1.5 References

The design and development of CyberSentinel draw upon a diverse range of academic papers, open-source projects, and practical guides. The following references inform the system's architecture and capabilities:

NSL-KDD Research

- *A Machine Learning Approach for Intrusion Detection System on NSL-KDD Dataset — academia.edu*
- *A Detailed Analysis on NSL-KDD Dataset Using Various Machine Learning Techniques for Intrusion Detection — academia.edu*
- *Intrusion Detection System with Customized Machine Learning Techniques for NSL-KDD Dataset — CMC, Tech Science Press*
- *Intrusion Detection System Using Random Forest on the NSL-KDD Dataset — Springer*

AI/LLM Security

- *HuntGPT: Integrating Machine Learning-Based Anomaly Detection and Explainable AI with LLMs — MDPI*
- *IDS-Agent: An LLM Agent for Explainable Intrusion Detection in IoT Networks — OpenReview*
- *AI-Augmented SOC: A Survey of LLMs and Agents for Security Automation — MDPI*

Open-Source Projects

- *Redxbeat, AI-Powered-Intrusion-Prevention-System (AI-IPS) — GitHub*
- *LakshayD02, Intrusion_Detection_System_Python — GitHub*
- *kirit1193, HIDPS — GitHub*
- *Noel9812, RNIDS — GitHub*
- *isaiah-harville, NIDS: Real-Time Network Intrusion Detection Framework — GitHub*

Practical Guides & Comparisons

- *How to Build a Real-Time Intrusion Detection System with Python and Open-Source Libraries — freeCodeCamp*
- *Snort Versus Suricata in Intrusion Detection — ResearchGate*
- *A Comparative Analysis of Snort 3 and Suricata — ResearchGate*

2. Overall Description

2.1 Product Perspective

CyberSentinel is architected as a dual-component, tightly coupled system that bridges low-level network operations with high-level web visualization. It is a completely self-contained product designed to run locally on the host machine it is protecting. The system is not distributed across multiple enterprise servers; instead, the network processing engine and the graphical dashboard application operate in tandem on the same machine.

The processing engine functions as the authoritative core, handling all intensive tasks including network monitoring, stateful traffic aggregation, machine learning inference, and firewall manipulation. The web dashboard, conversely, serves strictly as a decoupled, visual presentation layer. It possesses no direct access to the host's network interface or operating system configuration. The integration between these two distinct environments is achieved through a localized, unidirectional data bridge: a custom data stream hosted by the processing engine that continuously pushes updates to the listening dashboard. Furthermore, the system extends its perspective to the cloud by optionally logging historical data and aggregate statistics to a remote database, ensuring that threat records are preserved even if the local dashboard is closed.

2.2 Product Functions

The current implementation of the CyberSentinel product supports a comprehensive suite of distinct, observable functions. These functions operate sequentially to form the complete detection and prevention pipeline:

- ***Continuous Traffic Interception:*** *The system perpetually monitors the host machine's network interface, intercepting inbound traffic to identify source and destination addresses, ports, and communication protocols.*

- ***Stateful Flow Aggregation:*** *The system maintains an ongoing state of network connections, grouping individual, disparate pieces of traffic into cohesive network flows based on shared source addresses over a rolling time or volume window.*

- ***Statistical Feature Generation:*** *The system transforms raw traffic data into a structured mathematical profile required by the predictive model.*

- ***Predictive Threat Classification:*** *The system leverages a machine learning model to evaluate the mathematical profiles, classifying the aggregated traffic as either benign or assigning it a specific, recognized attack label.*

- ***Automated Active Mitigation:*** *Upon confirming a malicious classification that exceeds a noise-reduction threshold, the system autonomously interfaces with the host operating system to create a persistent firewall rule, blocking all further communication from the offending address.*

- ***Real-Time Data Broadcasting:*** *The system actively pushes formatted telemetry data—including threat event details and overall system statistics—to any connected dashboards using an integrated streaming server.*

- ***Remote Persistent Logging:*** *The system securely authenticates with cloud infrastructure and writes permanent records of all detected threats and rolling statistical summaries to a remote database.*

- ***Interactive Visualization:*** *The web dashboard renders incoming telemetry data into an intuitive command center, updating numeric statistic cards, animating protocol distribution charts, and dynamically drawing network topology graphs that visually highlight active attacker connections.*

- ***AI-Assisted Interpretation:*** *The dashboard provides a dedicated user interface drawer that maps specific machine learning attack labels to static, pre-written, plain-English explanations detailing the technical mechanism of the attack and the automated actions taken by the system.*

- ***Manual Administrative Override:*** *The dashboard includes actionable buttons that transmit requests back to the processing engine, instructing it to execute operating-system commands to delete specific firewall rules and restore access for previously banned addresses.*

2.3 User Classes and Characteristics

The primary user class for the CyberSentinel platform is the ***System Administrator / Security Analyst****. This user is typically a professional responsible for maintaining the security posture of the host machine or local network.*

Characteristics and Requirements:

- ***Administrative Privileges:*** *The user MUST possess full administrative rights on the underlying operating system. Without these privileges, the product cannot successfully operate, as it requires elevated permissions to monitor raw network traffic and to modify the system's built-in firewall rules.*

- ***Workflow:*** *The user's primary workflow is straightforward and requires zero technical setup. The user simply accesses the dashboard in a modern web browser and passively monitors the real-time visualizations. When the system highlights a critical threat and automatically blocks a connection, the user utilizes the dashboard to review the threat details, read the provided AI explanation to understand the attack vector, and make an informed decision on whether to maintain the automated block or manually restore access if it is deemed a false positive.*

2.4 Operating Environment

CyberSentinel is subject to highly specific and restrictive operating environment constraints due to its reliance on platform-specific utilities and dependencies.

- ***Operating System Dependency:*** *The system is* ***strictly bound to the Microsoft Windows operating system****. This absolute constraint is dictated by the active prevention module, which utilizes built-in Windows firewall utilities to enforce security rules. The active prevention functionality will categorically fail if executed on alternative operating systems such as Linux or macOS, which utilize entirely different firewall management architectures.*

- ***Network Monitoring Prerequisites:*** *To facilitate the low-level traffic interception required by the system, the host Windows machine must have compatible packet capture drivers installed and configured to allow the software to read traffic directly from the network interface card.*

- ***Runtime Environment:*** *The product operates as a deployed, standalone system service. Users interact with the system purely by accessing the dashboard via a standard-compliant web browser, with no manual installation or dependency resolution required.*

2.5 Design and Implementation Constraints

The architecture and implementation of CyberSentinel are shaped by several significant technical constraints:

- ***Synchronous Monitoring Operations:*** *The core traffic monitoring mechanism operates synchronously, meaning the primary process is entirely consumed by the act of watching the network interface. This design dictates that all subsequent actions—such as statistical extraction, machine learning inference, and firewall modifications—must be offloaded to secondary background processes to prevent the system from dropping incoming traffic during heavy analysis.*

- ***Static Model Dependency:*** *The system's intelligence is entirely reliant on static, pre-trained machine learning models. The current implementation does not support online learning, dynamic model retraining, or automated updates to the predictive algorithm. The accuracy and capability of the system are permanently fixed to the state of the model at the time of its initial training.*

- ***Rigid Statistical Requirements:*** *The predictive classifier strictly requires a precise, mathematically derived set of statistical features to perform inference. If the network traffic cannot be accurately parsed to generate this specific mathematical profile, the system is fundamentally incapable of classifying the flow.*

- ***Fixed Threat Vocabulary:*** *The system is permanently bound to the taxonomy and attack categorizations defined during the initial training phase of the machine learning model. The system is trained to recognize a specific, finite set of attack categories. It cannot arbitrarily identify novel, zero-day attack types that fall outside this pre-defined vocabulary; it will either misclassify them as normal traffic or assign them to the closest matching known category.*

2.6 User Documentation

The product is accompanied by essential user documentation aimed at facilitating successful operation.

- ***Product Manual:*** *The primary manual provides a high-level architecture overview detailing the data flow from traffic ingestion to visualization. It provides operational guidelines for accessing the dashboard, interpreting the visual data, and utilizing included simulation tools to safely generate local anomalous traffic, thereby allowing the user to verify the automated blocking mechanism without exposure to real-world threats.*

2.7 Assumptions and Dependencies

The successful deployment and operation of CyberSentinel rely on several critical assumptions regarding the host environment and external dependencies:

- ***Cloud Connectivity:*** *It is assumed that the host machine maintains a stable outbound internet connection to authenticate with cloud services and write telemetry data to the remote database. If this connection is unavailable, the system is designed to degrade gracefully, logging warnings locally while maintaining core detection and prevention functionality, assuming the local communication stream remains operational.*

- ***Local Network Port Availability:*** *The architecture inherently assumes that specific local networking ports are available and not bound by other applications. The system relies on a dedicated local port to host the essential real-time data bridge between the processing engine and the dashboard. If this port is occupied, the dashboard will fail to receive real-time updates.*

- ***Third-Party Integrations:*** *The system is dependent on an ecosystem of established third-party libraries for network manipulation, complex mathematical processing, cloud integration, and advanced graphical rendering for the user interface.*

3. External Interface Requirements

3.1 User Interfaces

The user interface for CyberSentinel is a meticulously designed, single-page web application. The design language heavily utilizes modern UI paradigms, specifically a dark-themed aesthetic characterized by translucent backgrounds, distinct drop shadows, and vibrant neon accents to convey a highly technical, cybersecurity "command center" environment.

The interface is logically partitioned into several distinct, interactive functional areas:

- ***Global Navigation and Status Bar:*** *A persistent top-level navigation component that prominently displays the product title alongside a highly visible, dynamically updating connection status badge. When the processing engine is offline, this badge renders in a muted tone indicating it is waiting for a connection. Upon successfully linking to the data stream, it instantly transitions to a vibrant active state.*

- ***Sidebar Navigation:*** *A left-aligned vertical menu providing tabbed access to various specialized views, including the Dashboard Command Center, the dedicated Threats Log, the Network Topology map, and the Artificial Intelligence Analyst interface.*

- ***Key Performance Indicator (KPI) Statistic Cards:*** *A prominent horizontal array of translucent cards occupying the top of the dashboard. These cards dynamically display the absolute count of total traffic analyzed, the total number of attacks detected, the current volume of addresses actively blocked by the firewall, and a calculated percentage representing the overall threat level of the network.*

- ***Real-Time Protocol Chart:*** *A specialized section rendering a live-updating donut chart. This visualization breaks down the distribution of intercepted network traffic by its underlying communication protocol, providing immediate situational awareness of traffic composition.*

- ***Live Threat Log:*** *A comprehensive, tabular data grid that acts as the primary historical record of malicious activity during the session. Each row corresponds to a single blocked threat, detailing the attacker's source address, the target destination address, the specific protocol utilized, the machine learning classification label, and the current firewall block status. Crucially, each row features interactive action buttons, including an option to launch the AI analysis and an option to trigger manual remediation.*

- ***Interactive Topology Graph:*** *A complex, interactive visualization rendering a force-directed graph that maps the logical connections between external addresses (represented as distinct nodes) and the host network. Nodes identified as attackers are visually differentiated and connected to the host via striking, red-dashed "attack vector" edges, providing a clear visual representation of network assaults.*

- ***AI Analyst Drawer:*** *A sliding side-panel interface that overlays the main dashboard when invoked by the user. This panel presents a highly structured, plain-English narrative of the selected attack, categorizing the information into Severity, Summary, Technical Mechanism, and Automated Action Taken.*

3.2 Hardware Interfaces

The software system directly interacts with the underlying hardware through the host machine's Network Interface Card (NIC). The processing engine utilizes specialized drivers to open connections bound directly to the NIC, allowing the software to bypass standard operating system network stacks and intercept individual pieces of data as they physically arrive at the hardware level.

3.3 Software Interfaces

CyberSentinel relies on two primary external software interfaces to fulfill its operational mandate:

- ***Host Operating System Firewall (OS Interface):*** *The most critical software interface is the direct integration with the Windows operating system's built-in advanced firewall. The processing engine programmatically executes system-level commands within a hidden shell to add and delete firewall rules dynamically. This interface is the sole mechanism by which the system achieves active intrusion prevention.*

- ***Cloud Database (Cloud Interface):*** *The system interfaces with remote cloud infrastructure to persist data. The backend authenticates via a local credential file to transmit document writes over secure connections to specific collections and documents within a NoSQL database. Concurrently, the dashboard establishes long-lived connections to these same documents, enabling real-time, remote data synchronization for users.*

3.4 Communications Interfaces

The system architecture necessitates highly efficient, real-time communication between the decoupled processing engine and the visualization dashboard. This is achieved through specific, localized communication protocols:

- ***Real-Time Event Stream:*** *The primary unidirectional data conduit is a custom-implemented event stream. The backend instantiates a local server, and the dashboard maintains a persistent connection to this endpoint. The server continuously pushes formatted text payloads down this open connection, instantly delivering new threat events and updated statistical counters to the user interface without requiring the dashboard to aggressively poll for updates. The server also transmits periodic heartbeat messages to ensure the connection*

remains active during periods of low network traffic.

- ***Application Programming Interface (API):*** *For bidirectional interaction—specifically, the manual restoration of access for blocked addresses—the server exposes a standard endpoint. The dashboard transmits a payload containing the target address to this endpoint. The server processes the request, executes the necessary operating system deletion command, and returns a standard response confirming success or failure.*

4. System Features

4.1 Real-Time Network Traffic Interception

4.1.1 Description and Priority

Priority: High. *The absolute foundational feature of CyberSentinel is its ability to continuously and reliably intercept live network traffic directed at the host machine. Without this raw data, the entire analytical pipeline cannot function.*

4.1.2 Stimulus/Response Sequences

Stimulus: *The deployed product is actively running as a system service.*

Response: *The system continuously monitors the primary network interface. As traffic physically arrives, the system parses the data, filtering it to ensure the destination address matches the host machine, and subsequently places the relevant traffic into a memory queue for further processing.*

4.1.3 Functional Requirements

REQ-F1-01: *The processing engine SHALL continuously monitor the host machine's primary network interface to intercept inbound network traffic.*

REQ-F1-02: *The system SHALL apply a strict filter to evaluate the destination address of intercepted traffic, discarding all data not explicitly directed to the host machine to reduce unnecessary computational overhead.*

REQ-F1-03: *The system SHALL instantly enqueue captured, relevant traffic into a mechanism that allows the primary monitoring process to continue uninterrupted while secondary processes handle complex analysis.*

4.2 Traffic Feature Extraction and Temporal Flow Aggregation

4.2.1 Description and Priority

Priority: High. *Individual pieces of network traffic rarely provide sufficient context to reliably identify complex cyber attacks. This feature aggregates disparate traffic into cohesive network flows and performs complex mathematical transformations to extract the specific statistical features required by the predictive model.*

4.2.2 Stimulus/Response Sequences

Stimulus: *The secondary processing mechanism retrieves raw, parsed traffic from the shared queue.*

Response: *The system evaluates the source address and either initializes a new flow state or appends the traffic to an existing flow. The system continuously evaluates the flow against configured temporal and volumetric thresholds. If the flow meets the criteria, the system calculates a complex mathematical profile.*

4.2.3 Functional Requirements

REQ-F2-01: *The system SHALL logically group incoming network traffic into discrete flows, uniquely identified by their source address.*

REQ-F2-02: *The system SHALL maintain the state of each network flow in memory until it reaches a statically configured volumetric threshold, ensuring sufficient data is collected for analysis.*

REQ-F2-03: *The system SHALL maintain the state of each network flow in memory until it reaches a statically configured temporal threshold, ensuring slow-rate attacks are eventually analyzed.*

REQ-F2-04: *Upon a flow reaching either its volumetric or temporal threshold, the extraction module SHALL calculate highly specific statistical features necessary to populate the machine learning input model.*

4.3 Machine Learning Threat Classification and Inference

4.3.1 Description and Priority

Priority: High. *This feature represents the cognitive core of the system. It replaces traditional, rigid signature-matching techniques with a probabilistic machine learning algorithm capable of identifying complex attack patterns across distinct categories.*

4.3.2 Stimulus/Response Sequences

Stimulus: *The extraction module successfully generates a complete statistical profile representing a completed network flow.*

Response: *The system passes the profile to the machine learning model. The model executes its logic, returning a specific classification label (e.g., 'normal', 'portsweep') and a flag indicating whether the classification represents a malicious threat.*

4.3.3 Functional Requirements

REQ-F3-01: *Upon initialization, the system SHALL successfully locate and load the pre-trained machine learning model and its associated data scaling pipelines into system memory.*

REQ-F3-02: *The system SHALL submit the calculated statistical profiles to the loaded model for predictive inference.*

REQ-F3-03: *In instances where a single flow generates multiple discrete profiles, the classification module SHALL implement a majority voting algorithm to determine the ultimate, highest-confidence classification label for the entire flow.*

REQ-F3-04: *The classification module SHALL categorize every evaluated flow with a distinct label and explicitly determine if the label constitutes benign traffic or a malicious attack.*

REQ-F3-05: *The system SHALL enclose all inference operations within robust error-handling routines to ensure that malformed data logs a warning rather than crashing the primary monitoring processes.*

4.4 Active Automated Firewall Blocking

4.4.1 Description and Priority

Priority: High. *This is the critical "Prevention" component of the product. It transitions the system from a passive observer to an active defender, automatically modifying the host operating system to sever malicious connections without requiring human intervention.*

4.4.2 Stimulus/Response Sequences

Stimulus: *The classifier definitively flags an aggregated network flow as malicious, AND the flow contains a minimum threshold of traffic.*

Response: *The system utilizes operating system utilities to silently execute a command in the background, instantly creating a persistent inbound block rule for the attacker's source address. The system then registers this address in a cache to prevent redundant system calls.*

Stimulus: *A user clicks the interactive "Unblock" button on a specific threat row within the web dashboard.*

Response: *The server receives a request, extracts the target address, and utilizes operating system utilities to execute a command that deletes the corresponding firewall rule, successfully restoring network access.*

4.4.3 Functional Requirements

REQ-F4-01: *Upon confirming a malicious classification that meets mitigation criteria, the system SHALL programmatically configure the host operating system's built-in firewall to block all inbound traffic from the offending source address.*

REQ-F4-02: *The system SHALL enforce a strict noise-reduction threshold, refusing to execute a block unless the malicious flow consists of a minimum volume of intercepted traffic, thereby minimizing false positives.*

REQ-F4-03: *The system SHALL rigorously evaluate the offending source address against a hardcoded whitelist of critical internal addresses, completely aborting the blocking operation if a match is found to prevent catastrophic self-isolation.*

REQ-F4-04: *The system SHALL evaluate the offending source address against a hardcoded list of essential cloud provider prefixes, aborting the blocking operation to ensure uninterrupted communication with critical infrastructure.*

REQ-F4-05: *The system SHALL maintain an active, in-memory list containing all currently blocked addresses during the session, checking this list prior to any firewall execution to prevent the redundant creation of duplicate rules.*

REQ-F4-06: *The system SHALL process manual override requests by executing the corresponding firewall deletion command and subsequently removing the address from the internal tracking list.*

4.5 Real-Time Threat Event Broadcasting

4.5.1 Description and Priority

Priority: Medium. *To ensure the web dashboard accurately reflects the current state of the network in real time, the processing engine acts as a localized streaming server, actively pushing telemetry data to connected dashboards.*

4.5.2 Stimulus/Response Sequences

Stimulus: *The processing engine completes the evaluation of a network flow, classifying it as either normal or an attack.*

Response: *The system constructs a data payload containing detailed metadata. It then pushes this payload down the open connection to all actively listening dashboards.*

4.5.3 Functional Requirements

REQ-F5-01: *The system SHALL host a localized streaming endpoint specifically configured to manage long-lived event connections.*

REQ-F5-02: *Upon the evaluation of any network flow, the system SHALL serialize an event payload and immediately transmit it to all connected dashboard clients.*

REQ-F5-03: *Every event payload SHALL include a nested object containing the absolute current system statistics, including the cumulative total of traffic processed, the total number of attacks identified, the total count of currently blocked addresses, and a calculated threat level percentage.*

REQ-F5-04: *The streaming server SHALL proactively transmit a minimal heartbeat payload to all connected clients periodically during periods of inactivity to prevent connection timeouts and ensure longevity.*

REQ-F5-05: *The server SHALL explicitly inject standard security headers into all responses to permit the dashboard application to establish the connection without violating browser security sandboxes.*

4.6 Persistent Threat and Statistics Logging

4.6.1 Description and Priority

Priority: Medium. *While the local stream provides immediate situational awareness, this feature ensures that all critical threat events and historical statistics are permanently preserved in a secure, remote cloud database for future auditing and analysis.*

4.6.2 Stimulus/Response Sequences

Stimulus: *The classifier definitively flags an aggregated network flow as a malicious attack.*

Response: *The system utilizes cloud integration tools to asynchronously construct and transmit a comprehensive threat document to the remote database collection.*

Stimulus: *The system updates its internal cumulative counters for traffic processed or attacks detected.*

Response: *The system utilizes cloud integration tools to overwrite the live statistics document in the database with the newly updated integers.*

4.6.3 Functional Requirements

REQ-F6-01: *Upon its first required invocation, the system SHALL attempt to initialize cloud integration by locating and authenticating with a local credential file.*

REQ-F6-02: *Whenever a malicious flow is identified, the system SHALL execute an asynchronous write operation, pushing a fully populated document—containing timestamps, addresses, ports, protocol, attack label, and block status—into a designated database collection.*

REQ-F6-03: *Whenever system counters are incremented, the system SHALL execute an overwrite operation on a designated, singular database document, ensuring the cloud database accurately reflects the current session's cumulative statistics.*

REQ-F6-04: *If the credential file is absent, or if initialization fails due to network connectivity issues, the system SHALL degrade gracefully, logging a descriptive warning locally and bypassing all subsequent database write attempts without disrupting the core local detection and prevention loop.*

4.7 Command-Center Dashboard and Real-Time Visualization

4.7.1 Description and Priority

Priority: High. *The web dashboard translates the raw telemetry data streamed from the processing engine into a visually compelling, highly intuitive graphical interface, empowering users to rapidly interpret and respond to complex network events.*

4.7.2 Stimulus/Response Sequences

Stimulus: *The dashboard receives an event payload via the open streaming connection.*

Response: *The application updates its visual state. The statistic cards instantly update their numeric values; the threat log prepends a new row detailing the attack; and the topology graph dynamically instantiates a new node for the attacker's address, drawing a striking red-dashed line connecting it to the central host node.*

4.7.3 Functional Requirements

REQ-F7-01: *The dashboard application SHALL prominently display Key Performance Indicators in visually distinct cards, reflecting the absolute values for Total Traffic, Total Attacks, Blocked Addresses, and a calculated Threat Level percentage as received from the stream.*

REQ-F7-02: *The dashboard SHALL implement a real-time, scrollable data grid that dynamically appends new rows for every detected threat, explicitly rendering the source address, destination address, protocol, classification label, and the current active firewall block status.*

REQ-F7-03: *The dashboard SHALL render an interactive "Unblock" button on every row representing a currently blocked address. Activating this button SHALL construct and transmit a request containing the target address to the local system's manual override endpoint.*

REQ-F7-04: *The dashboard SHALL render an interactive, force-directed topological map, visually plotting the central host machine as a distinct node and dynamically plotting all connecting external source addresses as peripheral nodes.*

REQ-F7-05: *The topology graph logic SHALL actively parse incoming threat data and explicitly draw visually alarming, red-dashed edges between the host node and any peripheral node identified as an active attack vector by the machine learning engine.*

4.8 Artificial Intelligence Threat Explanation

4.8.1 Description and Priority

Priority: Low. *To accelerate incident response and assist users, the dashboard incorporates an integrated Artificial Intelligence interface designed to translate obscure classification labels into clear, actionable, plain-English security narratives.*

4.8.2 Stimulus/Response Sequences

Stimulus: *A user clicks the interactive "Explain" button located on a specific threat row within the threat log.*

Response: *The dashboard smoothly transitions a side-panel drawer into view. The application cross-references the threat's specific label against its internal knowledge base, rendering a structured report that details the attack's severity, a plain-English summary, the underlying technical mechanism, and a confirmation of the automated mitigation actions taken by the system.*

4.8.3 Functional Requirements

REQ-F8-01: *The dashboard SHALL render a hidden side-panel drawer component that smoothly animates into the viewport when a user explicitly requests a threat explanation.*

REQ-F8-02: *The dashboard application SHALL maintain an internal, static dictionary mapping specific, known attack labels to highly structured informational objects containing fields for Title, Severity, Summary, Technical Mechanism, and Automated Action Taken.*

REQ-F8-03: *When an explanation is requested for an unknown or novel attack label not present in the dictionary, the system SHALL dynamically generate a logical fallback explanation, adjusting the reported severity and recommended actions based strictly on whether the system successfully executed a firewall block against the source.*

5. Other Nonfunctional Requirements

5.1 Performance Requirements

Throughput and Latency Constraints: *Performance metrics, specific benchmark throughput limits, and rigorous latency targets are expressly* ***not specified*** *within the current implementation of the product. The architecture, relying on synchronous packet capture, does not enforce explicit resource usage limits or guarantee real-time microsecond performance under extreme volumetric flood conditions.*

5.2 Safety Requirements

Physical or Critical System Safety: *Given the purely software-based nature of this network monitoring tool, there are no specific physical safety requirements, hazard prevention protocols, or critical life-safety certifications identified or mandated in the current implementation.*

5.3 Security Requirements

- ***Access Control:*** *Access control mechanisms are* ***not specified*** *in the current operational implementation. While foundational planning documents articulate a future intention to integrate authentication to secure an administrative panel, no such login screens, user session management, or role-based restrictions currently exist in the product.*

- ***Credential Protection:*** *The system mandates that sensitive cloud credentials required for database logging must be stored locally as a plaintext file. The product relies on standard deployment practices to prevent the accidental exposure of this credential file to unauthorized personnel.*

- ***Operating System Privilege Enforcement:*** *The product establishes a strict security boundary regarding its execution context. The system actively verifies its ability to manipulate system-level configurations; if it lacks appropriate administrative rights during operation, it will fail gracefully, generate descriptive warning alerts requiring elevated permissions, and refuse to execute unauthorized firewall modifications.*

5.4 Software Quality Attributes

Robustness and Fault Tolerance: *The processing engine is engineered for resilience against malformed network traffic. The entire machine learning inference pipeline is deliberately isolated within error-handling routines. This design ensures that if the model encounters mathematical anomalies or structurally invalid data, it will log the error and seamlessly continue monitoring without crashing the primary system. Similarly, the local streaming server is programmed to gracefully handle abrupt dashboard disconnections by silently discarding stale data queues rather than halting server operations.*

5.5 Business Rules

- ***Critical Infrastructure Safeguard Rule:*** *To categorically prevent the automated system from accidentally crippling the host machine by blocking vital local infrastructure, the system enforces a strict programmatic safeguard. Internal network gateway addresses and trusted cloud provider subnets SHALL never be targeted for firewall blocking, regardless of the severity of the classification they trigger. This rule is absolute and overrides all predictive logic.*

- ***Anomalous Noise Reduction Rule:*** *To prevent a high volume of false-positive firewall blocks triggered by stray, isolated anomalous traffic, the system enforces a minimum volumetric threshold. A detected and confirmed threat SHALL only trigger the execution of a blocking command if the aggregated network flow responsible for the threat consists of a predefined minimum volume of intercepted traffic.*

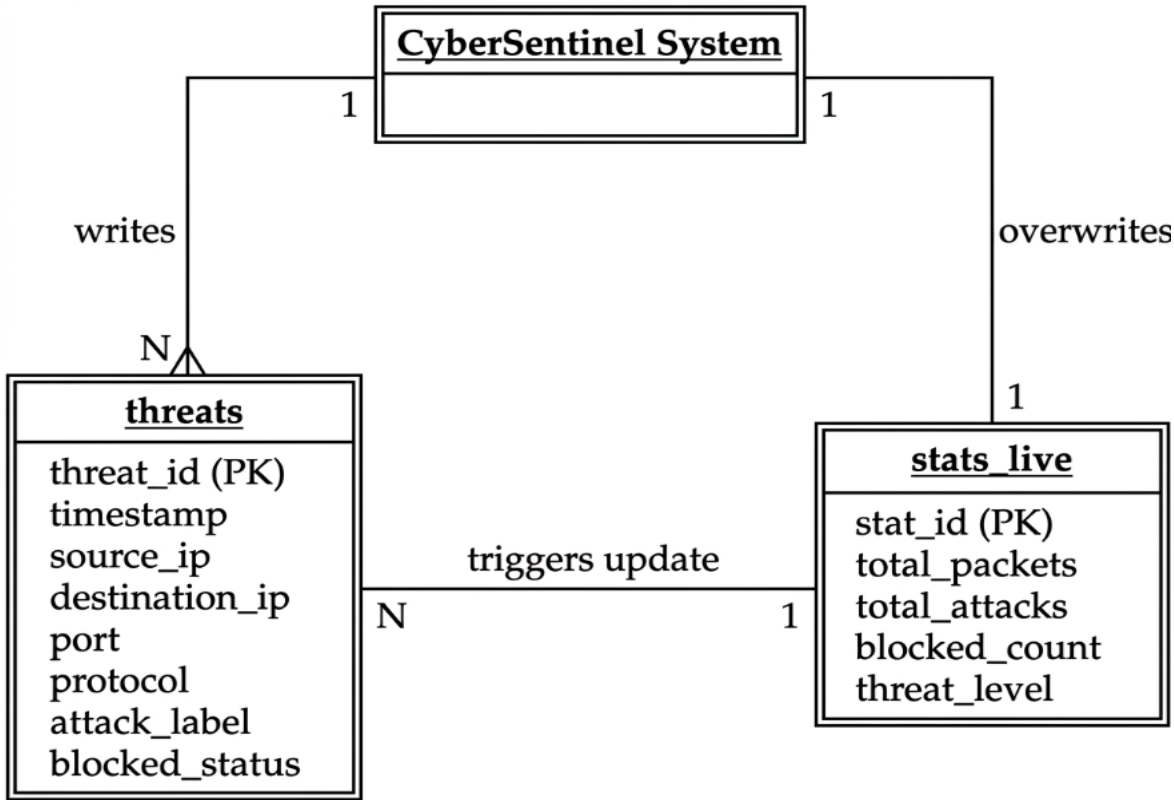
6. Other Requirements

Demonstration Capabilities: *To facilitate rigorous testing, evaluation, and demonstration without exposing the host to actual malicious actors, the product includes a standalone utility. This tool is capable of generating localized, simulated anomalous network traffic designed to safely and predictably trigger the entire detection, classification, and automated prevention pipeline.*

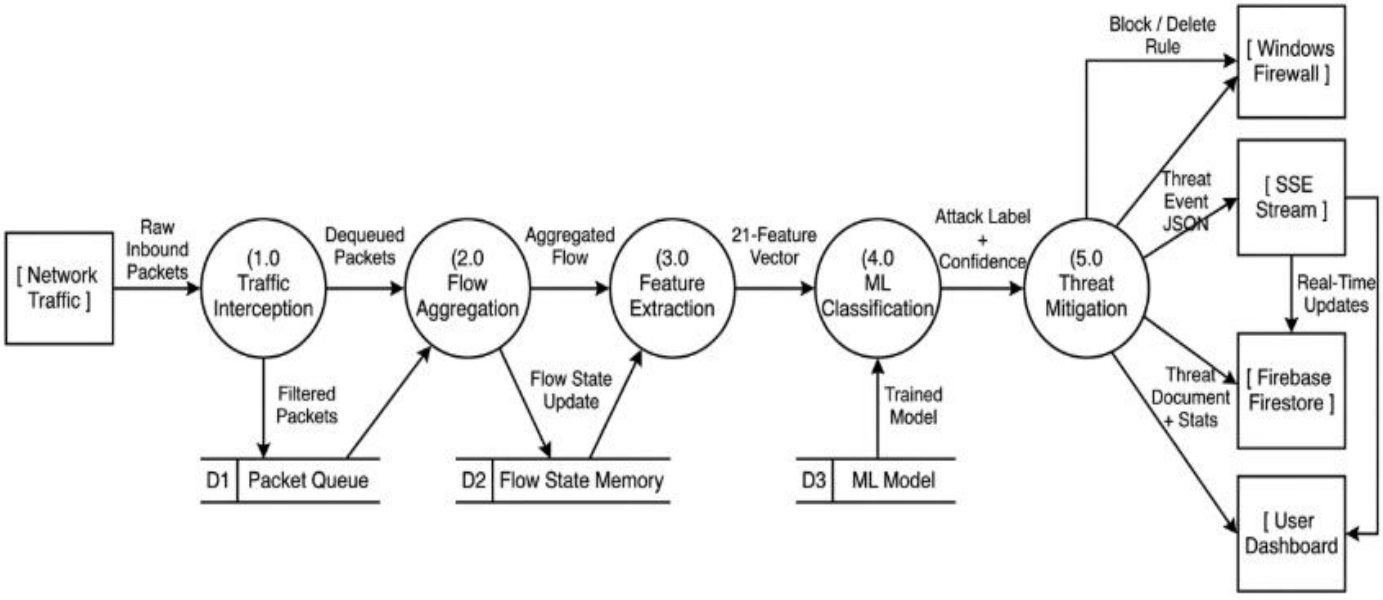
Appendix A: Glossary

- ***Flow:*** *In the context of CyberSentinel, a flow is defined as a logically grouped sequence of distinct network traffic that shares the exact same source address, aggregated over a specific, rolling time window or a specific volumetric count.*
- ***IPS (Intrusion Prevention System):*** *An advanced network security application that not only passively monitors network traffic for malicious activity but proactively and autonomously takes remedial action (such as altering firewall configurations) to block or prevent the identified threat from executing successfully.*

Appendix B: Analysis Models



Level-1 DFD



Appendix C: To Be Determined List

- ***TBD-1:*** *Section 5.1 - The definition and enforcement of explicit performance metrics, including acceptable benchmark throughput limits and maximum permissible latency targets for the automated firewall blocking response.*
- ***TBD-2:*** *Section 5.3 - The concrete technical implementation details of Authentication, session management, and Access Control intended to secure the dashboard interfaces, as previously proposed in foundational planning documentation.*
- ***TBD-3:*** *Section 3.1 - The formal definition of mobile responsiveness constraints, layout breakpoints, and strict multi-browser compatibility matrices for the web dashboard interface.*
- ***TBD-4:*** *Section 2.4 - The architectural redesign requirements, compatibility constraints, and the development of alternative active mitigation mechanisms necessary to port the system to non-Windows operating systems.*